Universidade do Minho

Licenciatura em Engenharia Informática - LEI

Computação Gráfica

Trabalho Prático - Fase 3

Grupo 14

A107324 - David José Barbosa Alves

A107368 - Diogo Malheiro Pais

A107377 - João Pedro Ferreira Gomes

A107332 - Pedro Afonso Quesado Pacheco Pereira

1. Introdução

O presente relatório descreve o desenvolvimento da terceira fase do projeto prático da unidade curricular de Computação Gráfica. O objetivo central desta etapa é a transição de modelos estáticos para cenas dinâmicas e animadas, introduzindo o conceito de tempo e trajetórias curvas, além de uma otimização crítica na performance de renderização.

As principais evoluções nesta fase focaram-se no **Engine**, nomeadamente:

1. **VBOs (Vertex Buffer Objects):** Substituição do *Immediate Mode* para uma gestão de memória de vídeo mais eficiente.
2. **Curvas de Catmull-Rom:** Implementação de trajetórias suaves para a animação de modelos no espaço.
3. **Animações Temporais:** Introdução de transformações geométricas (rotação e translação) dependentes do tempo.

2. Otimização: Vertex Buffer Objects (VBOs)

Nesta fase, o motor abandonou o uso de glBegin/glEnd, que enviava vértices para o GPU em cada frame, passando a utilizar VBOs para armazenar a geometria diretamente na memória da placa gráfica.

- **Abordagem e Decisões:** Durante o carregamento inicial (loadXML), os dados dos ficheiros .3d são agora carregados para *buffers* de vídeo. Cada modelo armazena o seu ID de buffer, permitindo que a renderização seja feita com um único comando de desenho.
- **Vantagem Técnica:** Redução drástica da latência de comunicação CPU-GPU, permitindo a visualização de cenas com milhões de triângulos (como sistemas solares detalhados) sem perda de *frame rate*.

Um desafio técnico significativo resolvido nesta fase foi a gestão rigorosa da Máquina de Estados do OpenGL. Garantimos o uso de glBindBuffer(GL_ARRAY_BUFFER, 0) e ativamos/desativamos o GL_VERTEX_ARRAY estritamente quando necessário. Isto preveniu conflitos de memória e artefactos visuais ("vertex spaghetti") ao intercalar o desenho de VBOs com o desenho imediato das linhas de trajetória (GL_LINES).

3. Curvas de Catmull-Rom

Para permitir que os objetos se desloquem em trajetórias curvas e suaves, foi implementado o algoritmo de splines de Catmull-Rom. Para manter o código do motor gráfico limpo e modular, toda a complexidade algébrica (cálculo de polinómios, derivadas e multiplicação de matrizes) foi encapsulada numa nova biblioteca independente (catmull.h e catmull.cpp).

- **Abordagem:** A trajetória é definida por um conjunto de pontos de controlo no XML. O motor calcula a posição do objeto interpolando entre estes pontos com base no tempo decorrido.
- **Algoritmo:**
    1. Cálculo do índice dos quatro pontos de controlo relevantes para o segmento atual.
    2. Aplicação da matriz de Catmull-Rom para obter as coordenadas (x, y, z).
    3. Cálculo do vetor derivada (tangente) para alinhar a orientação do objeto com a direção do movimento (eixo X do objeto segue a trajetória).

Durante a implementação matemática, deparámo-nos com um desafio visual onde os modelos sofriam distorções na escala e saltos anómalos. Através de *debug*, concluímos que a função de multiplicação de matrizes precisava de ser ajustada: foi crucial garantir que o vetor de Tempo (T) multiplicava pelas **colunas** da matriz de Catmull-Rom (e não pelas linhas). Esta correção assegurou trajetórias 100% contínuas e o cálculo correto das derivadas para a orientação.

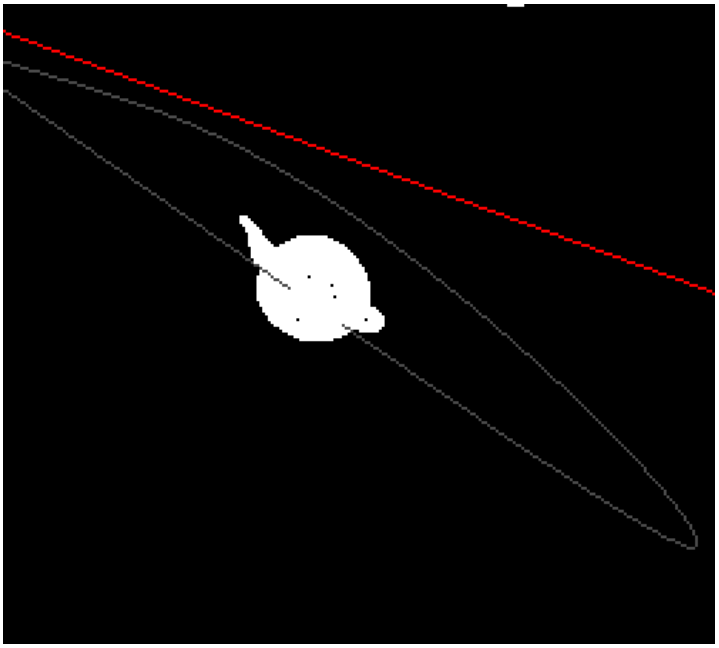

Figura 1: *Trajetória gerada por Catmull-Rom com detalhe do alinhamento do modelo (vetor tangente) em relação à curva.*

4. Geração de Superfícies de Bézier (Patches)

Como complemento ao motor gráfico, o módulo generator foi expandido para suportar a criação de modelos 3D complexos através de Superfícies de Bézier.

- **Abordagem:** O programa lê ficheiros com a extensão .patch, que contêm o número de "remendos", os índices e os pontos de controlo. Para converter estas superfícies matemáticas em geometria renderizável pelo OpenGL, implementámos os Polinómios de Bernstein.
- **Algoritmo:** Através de um nível de detalhe configurável (Tessellation), geramos uma grelha bidimensional (coordenadas U e V, de 0 a 1). Para cada ponto desta grelha, calculamos a sua posição exata no espaço 3D multiplicando os 16 pontos de controlo do patch pelos respetivos pesos de Bernstein. Finalmente, unimos estes pontos gerando dois triângulos por cada quadrícula, resultando em modelos de alta fidelidade como o famoso *Teapot*.

5. Animações e Scene Graph Dinâmico

O Scene Graph foi expandido para suportar tags de transformação baseadas no tempo, tornando o sistema solar verdadeiramente dinâmico.

- **Translação Animada:** Utiliza as curvas de Catmull-Rom para definir a órbita dos planetas e a trajetória do cometa. Seguindo as sugestões do enunciado, o modelo do cometa foi construído através de patches de Bézier utilizando os pontos de controlo do famoso *Utah Teapot*.
- **Rotação Animada:** Introdução da tag <rotate time="N" ... />, onde o objeto completa uma rotação de 360 graus em **N** segundos.
- **Herança Temporal:** Graças à estrutura recursiva e ao uso da *Matrix Stack*, as animações dos "pais" (ex: translação da Terra) propagam-se corretamente para os "filhos" (ex: a Lua mantém-se na órbita da Terra enquanto esta se move).

Para maximizar o realismo do *Scene Graph*, implementámos uma escala de tempo lógica (onde 1 segundo corresponde a 1 dia terrestre). Além disso, utilizámos uma técnica de composição de transformações: aplicando uma rotação estática imediatamente após a rotação temporal, introduzimos um *offset* angular. Isto permitiu espalhar os planetas pelas suas órbitas, evitando que o sistema solar iniciasse a simulação num alinhamento perfeito - fenómeno astronómico conhecido como **sizígia (syzygy)**.

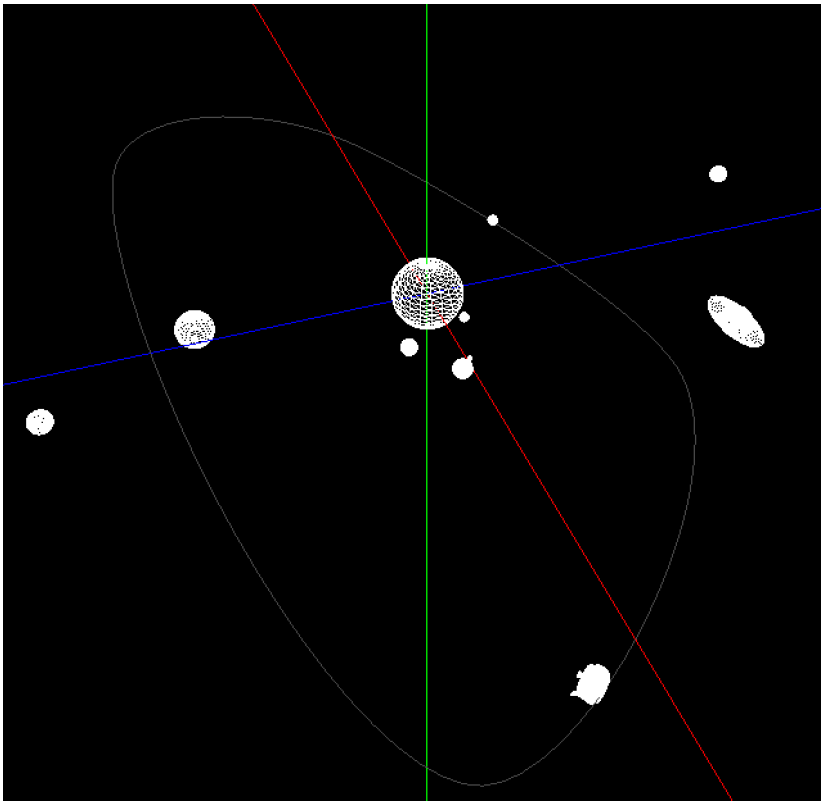

Figura 2: *Vista panorâmica do Sistema Solar dinâmico, evidenciando o offset angular inicial, diferentes raios de órbita e as inclinações axiais dos astros.*

6. Resultados e Testes

Nesta secção, validamos a robustez do motor através das cenas de teste da Fase 3.

- **Validação de Performance:** Comparação de FPS entre a Fase 2 (*Immediate Mode*) e a Fase 3 (VBOs), demonstrando a superioridade da nova arquitetura.
- **Fidelidade das Trajetórias:** Demonstração visual do movimento de cometas ou planetas seguindo curvas suaves, garantindo que não existem descontinuidades nos pontos de controlo.
- **Análise do XML:** Exemplo de como a nova estrutura do sistema_solar.xml utiliza os parâmetros time para simular as velocidades reais de rotação e translação dos astros.

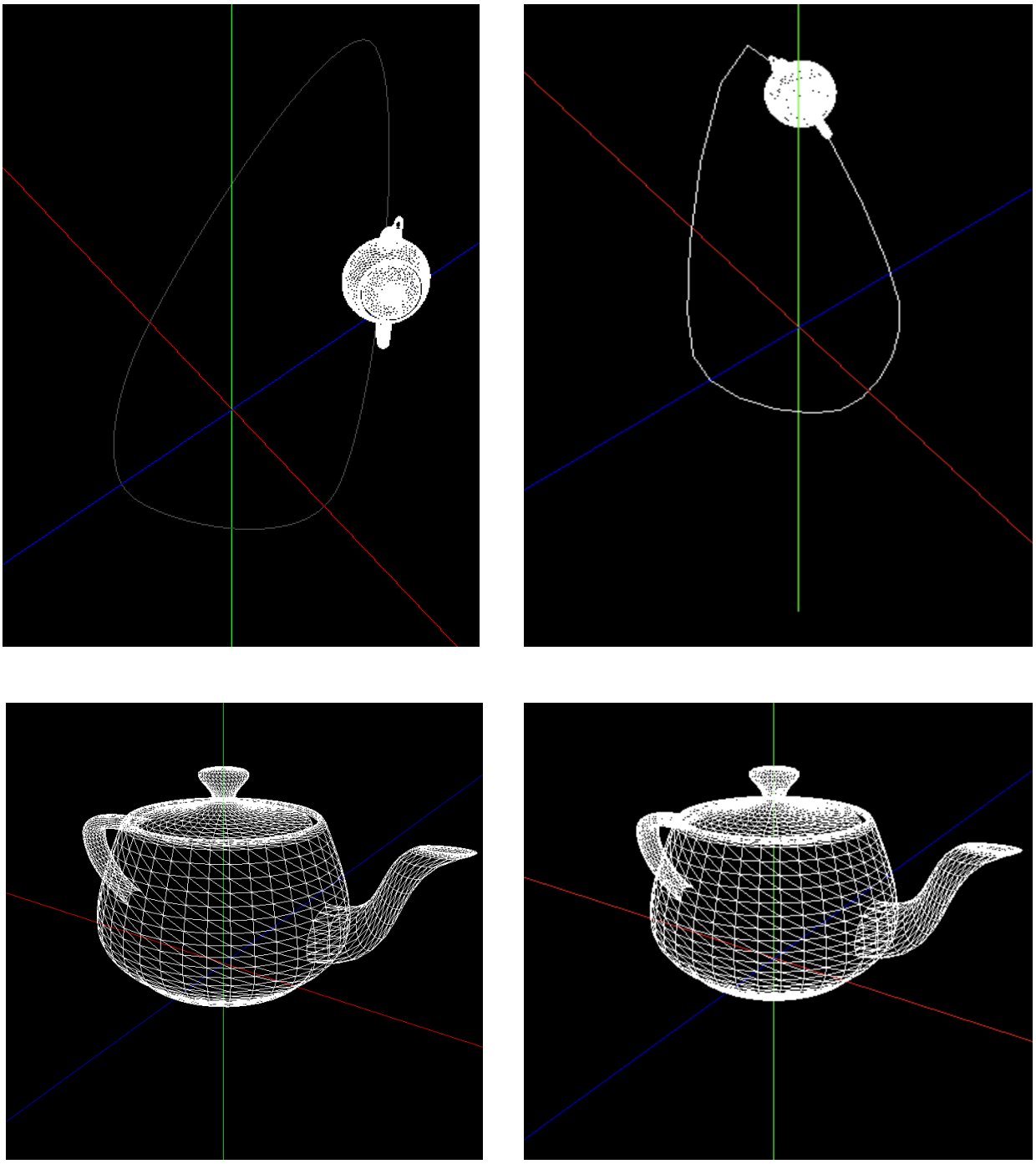

Figura 3: *Comparação direta entre a renderização do nosso motor (esquerda) e as imagens de referência fornecidas para a Fase 3 (direita).*

7. Extras: Navegação e Câmara Interativa

Ao encontro das funcionalidades de valorização propostas para o motor gráfico (Extra de 10%), a nossa implementação dispõe de uma câmara orbital interativa que permite uma exploração imersiva e total da cena gerada.

- **Controlo Dinâmico (Rato e Teclado):** Através das teclas **W, A, S, D** e da função de *callback* glutMotionFunc, o utilizador pode alterar os ângulos esféricos (α e β) em tempo real. O sistema regista o deslocamento do rato (Δ) e converte-o em variações angulares, recalculando em cada *frame* a posição da câmara através da conversão para coordenadas cartesianas:
    - $x = radius \times \cos(\beta) \times \sin(\alpha)$
    - $y = radius \times \sin(\beta)$
    - $z = radius \times \cos(\beta) \times \cos(\alpha)$
- **Sincronização e Reset (Tecla 'R'):** Para garantir a robustez da navegação, implementámos um algoritmo de sincronização no arranque que converte as coordenadas cartesianas do ficheiro XML nos ângulos internos da câmara. Adicionalmente, a tecla **'R'** funciona como um comando de *Reset*, restaurando instantaneamente o ponto de vista original de forma fluida e sem erros de posicionamento.
- **Gestão de Recursos:** O motor foi otimizado para procurar automaticamente os modelos na pasta dedicada /3d. Esta organização, aliada à atualização do ficheiro .gitignore, permite um repositório mais limpo, separando o código fonte dos ficheiros de dados gerados.

8. Conclusão e Trabalho Futuro

A terceira fase consolidou o motor como uma ferramenta capaz de gerar simulações complexas e fluidas. A introdução de VBOs e splines elevou a qualidade técnica do projeto para padrões modernos de computação gráfica.

Trabalho Futuro (Fase 4):

1. **Iluminação:** Implementação de fontes de luz (pontuais, direcionais e focais) e definição de normais para os modelos.
2. **Texturização:** Aplicação de texturas reais aos planetas para aumentar o realismo visual.
3. **Shaders:** Exploração de GLSL para efeitos visuais avançados.

9. Referências Bibliográficas

1. OpenGL Registry: "Vertex Buffer Objects (VBO) Specification."
2. Catmull, E., & Rom, R.: "A class of local interpolating splines."
3. LearnOpenGL.com: "Coordinate Systems and Transformations."